Chapitre 0 — Avant tout | Partie B — Le temps | leçon 0.12.3 sur 153 | difficulté 0 / 10

0.12.3. La fonction attendre() — attendre des secondes

#include <attendre> : attendre(2) arrête tout le programme deux secondes, puis il continue.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `attendre(secondes)` **arrête tout** le programme ce nombre de secondes, puis il continue à la ligne suivante. Simple, mais pendant ce temps rien d'autre ne se passe : la manette n'est pas lue. Pour attendre sans tout arrêter, il y a `chaque()`.

La ligne à écrire : `#include <attendre>`**.**

Son argument : le nombre de secondes, jusqu'à 255.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : ajoute une troisième ligne, TROIS, après un autre `attendre(1);`.

> **On retrouve attendre dans 3 leçons**
>
> 0.13, 0.13.1, 0.13.2

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <attendre> : attendre des secondes ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <attendre> : attendre des secondes ---- »). La console l'ignore.
`#include <attendre>  // attendre() : tout s'arrête un moment`	Demande à la console « attendre » (ce que c'est : attend des secondes entières). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « attendre() : tout s'arrête un moment ».)
`#include <texte>`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.

`texte(4, 6, "UN");`	Appel de la fonction texte de la console, avec les arguments 4, 6, "UN". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`attendre(2);  // deux secondes : rien ne bouge`	Appel de la fonction attendre de la console, avec l'argument 2. Ce qu'elle fait : attend ce nombre de secondes (jusqu'à 255), puis le programme continue. Pendant ce temps, rien d'autre ne se passe : la manette n'est pas lue. (Le commentaire le dit : « deux secondes : rien ne bouge ».)
`texte(4, 8, "DEUX");  // puis la suite`	Appel de la fonction texte de la console, avec les arguments 4, 8, "DEUX". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « puis la suite ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **332 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage          49 o  ← la console, à l'allumage  — la mise en route de la console
  main()             34 o  ← démarrage  — ta fonction
  VBlank              8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage      28 o  ← image()  — attend l'image suivante
  AttendreVBlank     16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces      12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte        10 o  ← texte()  — écrit un texte, lettre par lettre
  AttendreSecondes   17 o  ← attendre()  — attend des secondes entières
  CopierTuiles       58 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte       14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles             80 o  ← CopierTuiles  — 5 lettres sur 43
  "UN"                2 o  ← texte()  — un texte gravé
  "DEUX"              4 o  ← texte()  — un texte gravé
Lettres gravées : DENUX
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte()    × 2    22 o sur place, dans main()  + EcrireTexte (10 o)
```

```
  attendre() × 1     5 o sur place, dans main()  + AttendreSecondes (17 o)
  image()    × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <attendre> : attendre des secondes ----

#include <attendre>   // attendre() : tout s'arrête un moment
#include <texte>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

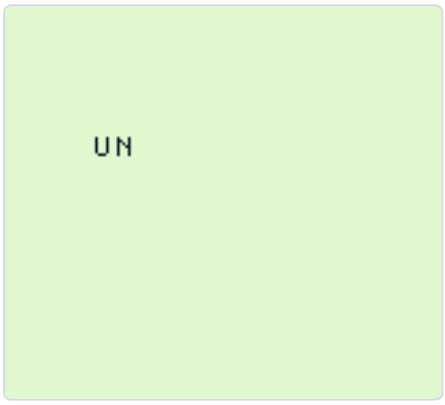
```
int main() {
  texte(4, 6, "UN");
  attendre(2);              // deux secondes : rien ne bouge
  texte(4, 8, "DEUX");      // puis la suite

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

> UN, puis DEUX deux secondes plus tard.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ UN est là tout de suite

✓ DEUX n'est pas encore là au départ

✓ deux secondes plus tard, DEUX est là

Ensuite : **0.13. Une lettre toutes les secondes, avec attendre** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++